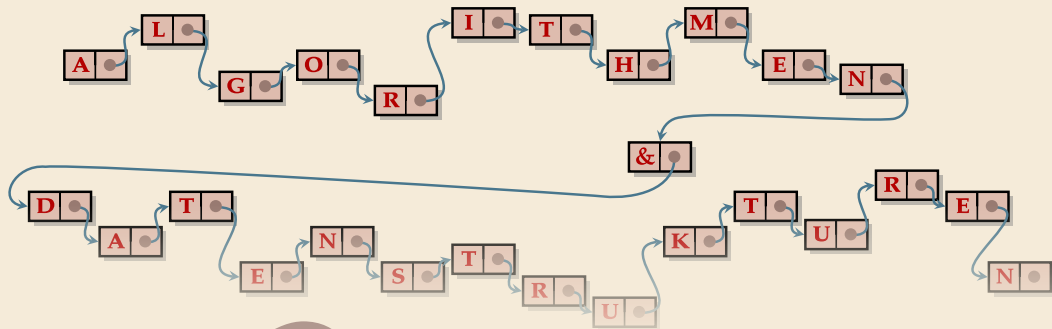




9

Sortierte Wörterbücher

Algorithmen & Datenstrukturen · Sommersemester 2026

Prof. Dr. Sebastian Wild

9 Sortierte Wörterbücher

- 9.1 Symbol Tables (Wörterbücher)
- 9.2 Konventionen in Java
- 9.3 Elementare Ideen für Symbol Tables
- 9.4 Priority Queues und Heaps
- 9.5 Operationen in binären Heaps
- 9.6 Binäre Suchbäume
- 9.7 Augmented BSTs
- 9.8 Balancierte Suchbäume: 2–3-Bäume
- 9.9 Balancierte Suchbäume: Red-Black-Trees

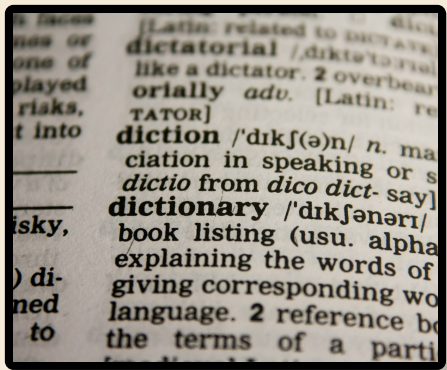
9.1 Symbol Tables (Wörterbücher)

Symbol Table ADT

Java: `java.util.Map<K,V>`

Symbol table (ST) / Dictionary / Map / Assoziatives Array / key-value store:

Python `dict {k:v}`



- ▶ `put( $k, v$ )` Python dict: $d[k] = v$
Füge key-value mapping (k, v) zur ST hinzu
- ▶ `get( $k$ )` Python dict: $d[k]$
Gib Wert für key k zurück (falls vorhanden)
- ▶ `delete( $k$ )` Python dict: `del d[k]`
Entferne key k (und assoziierten Wert) aus ST
- ▶ `contains( $k$ )` Python dict: k in d
Ist key k in der ST?
- ▶ `isEmpty(), size()`
- ▶ `create()`



Der wahrscheinlich wichtigste Baustein der Informatik!

(Jede Programmiersprache liefert eine ST mit.)

Symbol tables und mathematische Funktionen

- ▶ Symbol tables sind oberflächlich ähnlich zu (partiellen) mathematischen Funktionen
 - ▶ beide assoziieren (Funktions-) Werte zu Schlüsseln (Argumenten)
- ▶ aber: mathematische Funktionen sind *statisch/immutable* (unveränderlich; ändert Wert nicht)
(Eine andere Zuordnung von Werten ergibt eine gänzlich *andere* Funktion)
- ▶ Symbol Table = *dynamisches* Mapping
Also eine Funktion, die sich über die Zeit verändern kann

Basic symbol table API

Associative array abstraction. Associate one value with each key.

```
public class ST<Key, Value>
```

```
    ST()
```

create an empty symbol table

```
    void put(Key key, Value val)
```

put key-value pair into the table ← `a[key] = val;`

```
    Value get(Key key)
```

value paired with key ← `a[key]`

```
    boolean contains(Key key)
```

is there a value paired with key?

```
    void delete(Key key)
```

remove key (and its value) from table

```
    boolean isEmpty()
```

is the table empty?

```
    int size()
```

number of key-value pairs in the table

```
    Iterable<Key> keys()
```

all the keys in the table

Ordered symbol tables

Optional können Symbol Tables weitere Operationen unterstützen:

- ▶ $\text{min}()$, $\text{max}()$
Kleinster bzw. größter Schlüssel in ST
- ▶ $\text{floor}(x)$, $\lfloor x \rfloor = \mathbb{Z}.\text{floor}(x)$
Größter Schlüssel k in ST mit $k \leq x$.
- ▶ $\text{ceiling}(x)$
Kleinster Schlüssel k in ST mit $k \geq x$.
- ▶ $\text{rank}(x)$
Anzahl an Schlüsseln k in ST $k < x$.
- ▶ $\text{select}(i)$
Der i -kleinste Schlüssel in ST (Null-basiert, d.h., $i \in [0..n)$)

Wie beim Sortieren benötigen wir hier eine Vergleichsfunktion (Comparable oder Comparator)

Ordered symbol table API

```
public class ST<Key extends Comparable<Key>, Value>
```

...

Key min() *smallest key*

Key max() *largest key*

Key floor(Key key) *largest key less than or equal to key*

Key ceiling(Key key) *smallest key greater than or equal to key*

int rank(Key key) *number of keys less than key*

Key select(int k) *key of rank k*

void deleteMin() *delete smallest key*

void deleteMax() *delete largest key*

int size(Key lo, Key hi) *number of keys between lo and hi*

Iterable<Key> keys() *all keys, in sorted order*

Iterable<Key> keys(Key lo, Key hi) *keys between lo and hi, in sorted order*